

PercePT

Lincoln Olsen

Team Number 16

April 30th, 2026

Final Project Report for
HCI 575: Computational Perception

Iowa State University
Spring 2026

ABSTRACT

Millions of Americans attend physical therapy each year. Since the physical therapist can't attend each training session the patient does at home, they can't instruct the patient on the effectiveness of their training. PercePT is an application that can record a user conducting their exercises and send it to the physical therapist for comments on their progress and technique. A web interface is used to allow this communication, also providing tutorials on the movements with post-session summaries on the workout. PercePT gives them real time feedback as they are doing their exercises, letting them know if they need to improve the quality of their exercise session. The system uses MediaPipe for skeletal tracking, so the user's position is always known. A One Euro Filter is used for smoothing coordinating data, keeping the digital tracking stable and accurate. A Finite State Machine is utilized to keep track of the patient's current repetition. Just by using the camera on a patient's laptop, PercePT allows the patient to get real-time instructions on improving the quality of their exercise, along with detailed feedback from their physical therapist.

Introduction

Each year, roughly 50 million Americans must attend physical therapy [1]. This is due to a multitude of factors, such as surgery, stroke, pain, stiffness, and many more setbacks. In general, a physical therapist will teach the patient a few exercises at their first appointment, and it is the patient's job to do those exercises at home to build strength, since in most cases, the human body will not be able to heal itself at that singular therapy session. To ensure that the patients are doing their exercises to their fullest potential which maximizes recovery, PercePT allows for detailed feedback when no physical therapist is around to see them. PercePT is a live-video feedback tool that provides patients with a report on how well they are doing their exercise. To start, the patient starts the recording and uses PercePT's real time tracking feature to get an idea of how well they are doing their exercises currently. Although the web interface wasn't achieved in this project, the idea is that once their therapy session is over, the website uploads it so that the physical therapist can see how well the patient is doing and leave comments if necessary. The real-time video feed works by using MediaPipe, an open source google library that tracks the major joints on the human body. The program calculates joint angles to gauge how close the patient is to their ideal range of motion. A One Euro Filter is utilized to smooth the data received from MediaPipe, and a Finite State Machine keeps track of the whole workout process. This combination of tools allows patients to speed up their recovery with detailed feedback from both PercePT and their physical therapist.

Why use PercePT at all?

While physical therapists can view their patients' exercises, this platform is made for anybody who is doing physical therapy for a condition that has a range of movement. For example, someone post-ACL surgery may be bending their leg to maximize range of motion and need to hit a 135-degree angle for fastest recovery. PercePT will track that by recording it, calculating angles, and responding to the user with either green, yellow, or red text on the screen denoting how closely they are bending that leg to 135-degrees. It is important to note that since PercePT is a tool that views various joints on the human body to function, this only helps with exercises that require movement. If someone is recovering from a stroke, PercePT has no understanding of helping with speech recovery but could help with basic bodily functions such as walking or sitting. Looking at the importance of PercePT from the physical therapist's perspective, there are 225,350 physical therapists in America in 2021 [2]. Along with this, there are roughly 38,000 physical therapy clinics spread across the United States [3]. There is no question that physical therapy is a dense and demanding field. Taking the 50,000,000 million US physical therapy patients from earlier and dividing it by the available 225,350 therapists yields 222 unique patients for one therapist over the course of a year. Although 38,000 clinics seem as if every patient in America has access to a clinic and therapist, patients in rural parts of the country must drive long distances to meet with a therapist. With PercePT's local capabilities and ability to allow anyone to show their therapist their progress, this saves tremendous time in travel and recovery time by getting real-time feedback as well as detailed feedback from their therapist online.

Related Work

In the commercial world, Kemtai seems to be the closest working software similar to the idea for my project. Although I could explain what they do, this two sentence summary from their page

seems to sum it up well: “Kemtai is a digital computer vision exercise platform focused on providing the most personalized and adaptive home exercises experience, for everyone. Kemtai helps physical therapists extend their reach and complement their in-clinic patient work by assuring patients are performing exercises correctly when at home alone and sharing the patient’s performance with the physical therapist” [4]. Kemtai’s goal is to ensure that patients are getting the most of their therapy sessions, but also provide that physical therapist to patient connection. This project implements the video-feed, but in a commercial setting, incorporating this into a user interface provides a full end-to-end system similar to Kemtai. Seeing Kemtai achieve such a high level of success using only a laptop camera shows that my project has potential to be expanded upon and be a major accomplishment. Although it was not possible to finish the interface between the therapist and patient, this project created the most integral part, which allows the user to get real-time feedback, with the web application easily being created later. On Kemtai’s website, they go in depth on how they use computer vision to track 111 data points on the human body to accurately see how well the exercises are being conducted, as well as giving a demo of what the software looks like mid-session [5]. Along with this, they start with a range of motion assessments using a mix of tools such as goniometer, inclinometer, and artificial intelligence to understand the patient’s body, which are ideas that I have explored throughout the project, using them to inspire different parts of the system [6]. Finally, there is an exercise library containing more than 2,000 videos on different exercises that a patient might need to mimic for their recovery plan [7]. This is a great point of reference for how I planned on styling the Exercise Tutorials screen if the web application were to enter development.

A paper published in 2026 wanted to prove that basic MediaPipe skeletal tracking was accurate and encouraged fitness. Out of 216 participants that did three 30-minute weekly sessions of seven compound exercises, it was found that the AI was 97.2% accurate in judging correct repetitions, proving that an application running MediaPipe and providing live updates can run on a lightweight device such as a phone [8]. In a similar study, Mediapipe’s ability to judge angles was tested. Dill et al. took MediaPipe and graded it against an 11-camera motion capture system. Correct and incorrect squats were performed in the experiment to test how well MediaPipe captured the angle compared to the 11-camera system. The researchers found that when the landmark data for MediaPipe was captured using a two-camera system with one in front and one on the side of the test subject, it achieved a 3D joint tracking error of 30.9 mm. It calculated this using the Root Mean Square Error which gives the researchers an idea of the average amount they were wrong by. This result means that when MediaPipe guesses where the subjects joint was, it was a little over an inch away from the exact position on average. Along with this, when the subject bent their knee, it was calculated to be 10.7 degrees off from its true position, which the researchers confirm is acceptable since this is closely related to a physical therapist. This study also mentioned that MediaPipe might have a built in Kalman Filter since they applied one and it didn’t have a huge impact on the study [9]. Even though it was found that the built-in Kalman Filter may be enough, the One Euro Filter was still implemented due to jittery data returned from MediaPipe. This study confirms that MediaPipe is a reliable tracking software that is useful for my project. Although it doesn’t compare to the 11-camera motion capture system, its availability and cost make it the best option for this project.

A huge part of this project is the dataflow between MediaPipe and OpenCV. In a study published by Asmawi and Kumaresan, OpenCV and MediaPipe were integrated together to see how well

squats were detected in a home workout environment. The process is similar to what PercePT uses, where joints were detected and the angles of the joints were calculated. The system detected squats 90% of the time and lunges 86% of the time out of five repetitions for each user. The study also mentions that lighting conditions play a significant factor in how well the system does, which is something that was kept in mind for PercePT, but was never an issue [10]. The following shows the screen display they used for the setup, like how PercePT uses green, yellow, and red for monitoring:

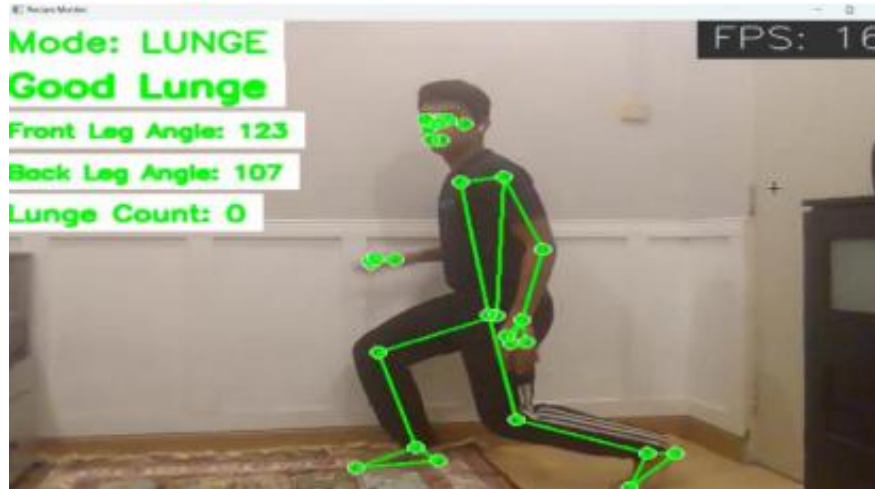


Figure 1: User performing a lunge with the MediaPipe landmarks overlaid. Image Credit: Asmawi and Kumaresan [10]

This is a great and influential example of what PercePT's screen looks like. The repetition count is displayed to screen, along with the angles of the limbs that affect the workout. This is quite a complex example compared to an exercise like a squat, where the bending of the legs is similar and that is the only angle that matters.

BlazePose is a key factor in the success of PercePT. Yang, Wang, and Shi of Zhengzhou University aimed to solve the same issue that PercePT solves: guide users during their rehabilitation workout to shorten their recovery time. The goal was to achieve this on a phone camera which is accessible to most people. They took the video feed, fed it to the BlazePose model, and fed the posture feature data generated by BlazePose into the posture analysis module for testing. Since BlazePose is a neural network created by Google for pose estimation, they were confident in what the results would be. After making three people do 200 total push-ups with 100 being done correctly and the other 100 done with poor form, here is the resulting table [11]:

Confusion Matrix		Prediction	
		Positive	Negative
Reference _	Positive	262	38
	Negative	29	271

Figure 2: Table showing pushup count because of BlazePose. Table Credit: Yang, Wang, and Shi [11]

Looking at these pushup numbers, BlazePose is clearly a reliable model for this project. What this article proved was BlazePose's ability to function well on low-level hardware. The researchers chose the Google Pixel 2 smartphone to operate the tests, which has far less computing power than a laptop. Since OpenCV, MediaPipe, and BlazePose are strung together correctly, the system will work efficiently. An important part of PercePT's success is the One Euro Filter. Its effects are best represented in a study done by Casiez, Roussel, and Vogel. They recorded a mouse cursor at 60 HZ move around a screen for around an hour to simulate accurate, smooth cursor movements. Then, they added white noise, ran the data through a filter and recorded the results. The following was found, with each test being represented by a color in the legend:

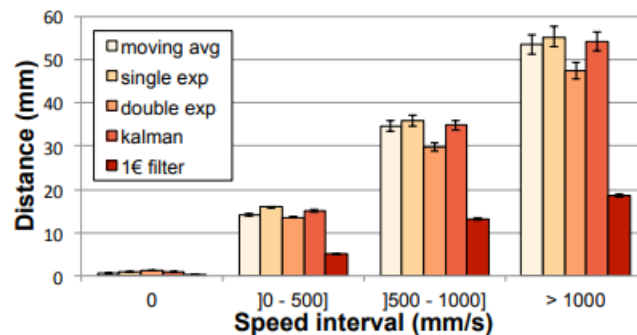


Figure 3: Results of various filters applied to a lagging cursor. Image Credit: Casiez, Roussel, Vogel [12]

The testing phase moved the cursor around at a certain speed and checked how far away the measured distance was from the original position to see the lag. Clearly, the One Euro Filter did an incredible job being at least twice as accurate in all tests compared to the other filters applied [12]. Something that was kept in mind is that MediaPipe seems to have a built in Kalman filter that is efficient, so it might not be worth the time to implement the One Euro Filter and could instead put the time somewhere else [9, 13]. During implementation, since some of the data returned by MediaPipe was noisy, the One Euro Filter was still implemented.

One of the biggest inspirations for this project was a study published in 2025 by Mishra et al. In this study, they used MediaPipe and OpenCV together to display a skeleton onto a human body. To accomplish this, they used a few extra python libraries such as PyGame, Pandas, and Streamlit that weren't key in PercePT's success, but similar libraries were used. They provided the functions they used, with most of them calculating the angle of every leg, as well as the neck and abdomen. Their angle calculation was done extremely well, where they built their own function to tell which body part was being used, and sent the code to the respective function to calculate the angle. This was their usage of a Finite State Machine, which is a functionality I heavily utilized in this project. They also use functions to represent what type of exercise is being conducted for further specification, such as `squat(self, counter, status)`, and `walk(self, counter, status)` which is an intelligent way to narrow down how the angle calculations will be done. To detect the body part, they use a DataFrame object from the Pandas library to keep track of the body part recognized. In their results, they found that the accuracy of their program had an average of 92% [14]. Due to this team's wonderful documentation, I referenced this paper heavily for my ideas and information consistently throughout the project, mainly due to its accuracy in comparison to PercePT. The accuracy of 92% might seem low, especially when the average person would think a physical therapist would get a 100% accuracy rate, but a study

done in 2023 proves that is not the case. Biebl et al. conducted a study to test two things: first, whether two physical therapists agree on a repetition's full completion, and second, whether a therapist and computer vision agree on a repetition's full completion. It was found that the two physical therapists agreed on the repetition's completion from the test group 83.3% of the time, while the physical therapist and computer vision agree 82.8% of the time, which is not much of a difference [15]. While the computer vision agreement was much less, this was a key study to show that PercePT is just as accurate as a physical therapist.

Previous Experience

This class is my first true experience with computer vision. I have been on teams that have used OpenCV but never been the one working on that aspect of the project. I am confident this project worked well and was finished for two reasons: first, I have had plenty of experience in Python which is the only language used in this project. There are many Python APIs that I have worked with, which is why I believe the OpenCV integration will be seamless. Second, looking at course work, I have taken embedded systems and multiple application development classes that taught me many principles which I know will apply throughout this project.

Software and Algorithms

Python/NumPy: The programming language that is used with this project is Python. Due to Python's seamless integration with API's and access to libraries, Python is a common language used for computer vision. NumPy (which stands for Numerical Python) is a Python library that is responsible for working with the millions of data points/pixels that need to be processed. Since NumPy's mathematical operations are written in C, it processes pixels much quicker than any code within a for loop would [17]. The primary Python libraries that haven't been mentioned that are used in this project are Tkinter [21], Time [22], and Matplotlib [23].

OpenCV: This is the leader in image processing functionality. OpenCV is an open-source library that contains hundreds of different computer vision algorithms within. The libraries I used are videoio for video capturing, imgproc for image filtering and drawing the skeleton to screen, and highgui to have an advanced GUI displayed to the screen [18].

MediaPipe: This is a highly trained and developed machine learning library that can track a patient's movements by putting 33 coordinates on their major joints. It is an open-source Google project. It will process the live video feed that is provided by OpenCV and utilizes the BlazePose model to map the 33 coordinates to the patient's joints. It provides the raw data needed to process what the video feed is seeing. It is important to note that MediaPipe uses two different models: the first is the pose detection model which is used for detecting the body with key landmarks, and the second is the pose landmarker model which adds the 33 landmarks to the joints of the human body [13]. The following is an image of what the coordinates look like on the patient:

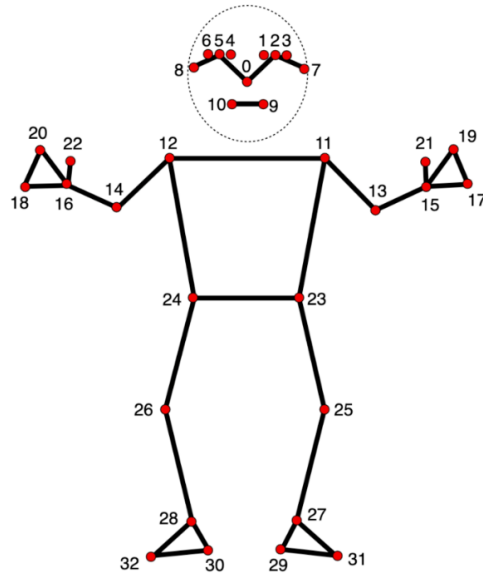


Figure 4: Visualization of the 33 landmarks tracked on the patient's body. (Image credit: Google [13])

One Euro Filter: This is a filtering algorithm used to eliminate jitter and noise. It looks specifically at coordinate data and reduces the jitter and lag, which is very useful when combined with the raw coordinate data received from MediaPipe. This is what keeps the video feed stable and accurate.

BlazePose: This is the core tracking model that runs inside MediaPipe. OpenCV handles the raw video feed, hands it to MediaPipe, and this neural network analyzes the pixels from the data and adds the 33 landmarks to the joints. While MediaPipe is the framework that takes the image from OpenCV and processes it, BlazePose does the dirty work by estimating the best spots on the human body for the 33 joints.

Tkinter: A Python library that simulates a GUI, this is utilized in the project to allow the user to pick their exercise, enter the degree angle they want that exercise to achieve (which would be provided by their physical therapist), and enter the amount of repetitions and sets they wish to accomplish. This will act as the screen before the live video feed, allowing the user to easily configure their exercise without having to do so in the code.

Finite State Machine: This is not as much an API I integrated into the project, but instead a logical structure I built into the code to keep track of the state the user is in during their workout. This uses Python logic, including a variable to keep track of the state and match/case statements depending on what the user is doing.

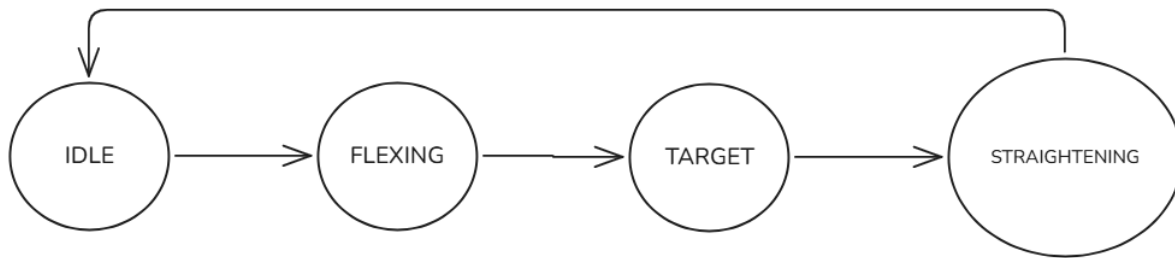


Figure 5: Finite State Machine for user movement. Image created using <https://excalidraw.com/>

The FSM is broken up into four stages:

IDLE: The user is in the resting state of the repetition. This is where they always start out.

FLEXING: Even though there are many different movements a user could be doing, the flexing state represents the user going from rest to their ideal target, which could be a straight leg to a 135-degree angle bend on their leg.

TARGET: The user has pushed the body part they are training to the limit for that repetition. This is where the text turns green in the ideal scenario to denote a good repetition.

STRAIGHTENING: This state represents the user going back to their initial rest state. For the example of bending their leg, this would be straightening it back to a straight leg.

The original plan was to use switch statements to make the code cleaner instead of if/else statements, but it was found that if/else statements were structurally easier to read in this project compared to switch statements. It is important to keep in mind that every state must have a finite and non-empty set of states, transition function, initial state, and final state set, which is what the current Finite State Machine has [19].

Equipment

The primary piece of equipment used is a laptop. The specific model I used for the development of this project is the MSI Modern 15 H AI C1MTG. Baggy clothing worked just as well as tight clothing in this project due to MediaPipe's ability to find the joints on the patient's body. Since this project is running computer vision models along with MediaPipe, it is recommended to use a machine with roughly 16 GB of RAM and a processor with a minimum of 4 cores to ensure smoothness. If a webcam is not available, a user should use a camera that has a resolution of at least 720p and 30 frames per second.

Methodology

All of the tools listed in the Software and Algorithms section were key for the completion of this project. The code was created in Python, utilizing OpenCV for computer vision, MediaPipe for skeletal tracking, the One Euro Filter for smoothing of coordinate data, BlazePose to assist

MediaPipe in skeletal tracking, Tkinter for the GUI, and a Finite State Machine for general logical flow.

The system is a Python-based desktop application utilizing the Tkinter Graphical User Interface to manage user input. Once the user inputs their necessary information such as target angle, sets, and repetitions, they can complete the workout with the video feed. While the Tkinter GUI is not directly a part of the computer vision aspect of the project, it provides a baseline interface that makes the application feel whole. It is also easy to implement, which is why it was used.

OpenCV captures live video of the patient doing their exercise. On the screen, the patient sees the skeleton from MediaPipe tracking their movements, but in the background, OpenCV is sending each frame to MediaPipe. The neural network operating within MediaPipe is analyzing the pixels in the image to calculate x, y, and z coordinates for 33 skeletal landmarks. OpenCV uses its drawing API's such as `cv2.putText` to write text to the screen, informing the patient of their data, such as their angle they are achieving as well as the sets and repetitions. It is important to mention that MediaPipe keeps a tracker for each landmark of how confident it is that it can see the joint, which is on a scale of 0.0 – 1.0. In the code in Figure 6, under the `process_landmarks` function, the first if statement is checking to ensure that MediaPipe is at least half-confident in its ability to see the key joints.

Along with receiving the x, y, and z coordinates of the joints, the system calculates the angle between the landmarks. This is immediately returned to the screen to show the user what the angle of their exercise is. Originally, a one euro filter was questionable to be implemented [9, 13] but due to the angle's willingness to jump around at small movements, the filter was implemented, giving the user much more control over finding the precise angle.

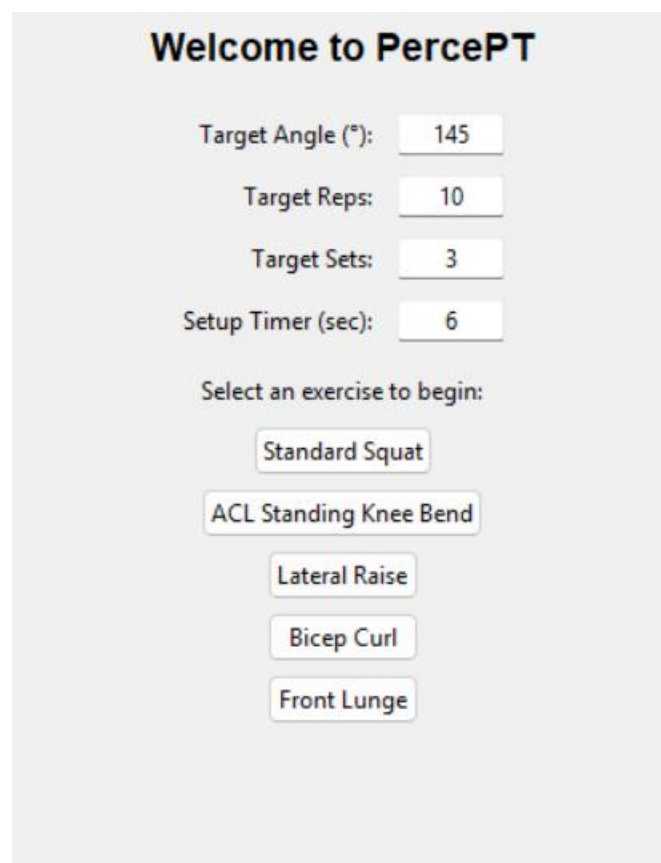
The Finite State Machine's job is to track what state the user is currently in. It will track their idle state, their flexing state, target state, and their straightening state. Once the exercise gets to a certain degree angle, it switches from the idle state to the flexing state. For example, during the exercise where the user bends their leg, their leg should be a straight line, which is 180 degrees. Once the user starts bending their leg and it gets to a 170-degree angle, the FSM now knows to switch to the flexing state. Once the user hits the exact degree angle they entered in the Tkinter GUI, the FSM will then be in the target state. If the user is short of the target state and starts to go back to their starting state, the target state will be skipped and the FSM will immediately go from the flexing state to the straightening state. Once the user crosses back over the original idle to flexing state degree angle (for example, in the case of bending the knee, this would be 170 degrees as mentioned earlier), the go back to the idle state and the loop repeats.

The architecture of the software was meant to replicate Object-Oriented Principles, where a class could be created to represent an exercise and be seamlessly integrated into the system. This would require creating a new Python file and implementing the methods from the parent class to simulate a new exercise.

Software Architecture

The entire codebase is written in Python, primarily utilizing Python libraries for features throughout the project. Basic Object-Oriented Programming principles are used. The following are the main classes:

main.py: Similar to most main.py files in a project, this is where all of the methods and classes are called and ran. All of the following files that will be named play a part in main.py. The Tkinter library acts as the GUI for this project. As it is shown in Figure 6, a user starts by entering their target angle, target reps, and target sets. It is assumed that all of this data is provided by the physical therapist. Following those inputs, the patient can select the workout that they want to do. Selecting one of these options takes the user to the camera vision aspect.



The screenshot shows a Tkinter window titled "Welcome to PercePT". It features four input fields for user configuration: "Target Angle (°)" with the value 145, "Target Reps" with 10, "Target Sets" with 3, and "Setup Timer (sec)" with 6. Below these fields is a label "Select an exercise to begin:" followed by five buttons: "Standard Squat", "ACL Standing Knee Bend", "Lateral Raise", "Bicep Curl", and "Front Lunge".

Figure 6: PercePT starting screen

exercise.py: This is the parent class for every exercise class. There are currently five child classes that extend this class. There are three methods, the first one being `calculate_angle.py`. Since the skeleton drawn by MediaPipe will always contain the same logic, the manner in which the program calculates an angle will never have to change. This method takes three parameters, a, b, and c. The method calculates the 2D angle between three points, and returns the angle. The following formula is used to calculate the angle: $\arctan2(c[1] - b[1], c[0] - b[0]) - \arctan2(a[1] - b[1], a[0] - b[0])$. This formula does the following:

- The formula uses arc tangent to calculate the angle in radians between the b to c vector in the line $\arctan2(c[1] - b[1], c[0] - b[0])$, similarly this is done for $(a[1] - b[1], a[0] - b[0])$ for the b to a vector. The X and Y coordinates are subtracted from each other to find the angle. Then, the angle from b to a is subtracted from the angle of c to b to find the actual angle of the joint. After this, the angle is converted from radians to degrees by multiplying by $\pi/180$, shown in the following line from the code: `angle = np.abs(radians * 180.0 / np.pi)`
- Arc tangent is used since the joints give the opposite and adjacent side lengths. For example, connecting the hip to the knee, if a straight line horizontally is drawn from the hip and a vertical line is drawn from the knee, a right triangle is formed. The hypotenuse is the bone connecting the joints. Since MediaPipe gives the X and Y coordinates of the joints, subtracting the coordinates can give the opposite and adjacent sides, making the tangent easy to find. Arc tangent is specifically used to get a turn a ratio generated by tangent into a measurable angle.

The next two methods do not contain any code, they are both abstract methods. The first is `process_landmarks`. Since different parts of the body will require different landmarks (i.e. bicep curl needs to know where the elbow is and squat should know where the hip is), this method is implemented in each subclass to allow the program to understand what part of the body it is looking at. It takes in the skeleton given by MediaPipe and ensures that all three given landmarks are visible (for example, this could be ankle, knee, and hip for a squat). The algorithm takes the x and y coordinates of the given joints and calculates the angle formed by the movement. The One Euro Filter is also utilized here to filter out the noise in the data. The angle is then returned.

Finally, the `update_fsm` function is used as the finite state machine for each exercise. This keeps track of the state that the user is in during their exercise. It does this by taking the angle of the exercise that they are doing, and classifies it into four different states: IDLE, FLEXING, TARGET, and STRAIGHTENING. These states are further explained in the Software and Algorithms section of this report. Both methods are shown in Figure 7. The methods are from the `squat.py` class, representing how these two methods are structured in every subclass of `exercise.py`

squat.py, bicep_curl.py, front_lunge.py, lateral_raise.py, standing_knee_bend.py: These files hold the specific requirements for each exercise. As it was mentioned, this is where the Finite State Machine logic is created for each respective exercise. Also, the `process_landmarks` function is implemented differently for each exercise since not every exercise requires seeing the same joints. Objects of these classes are instantiated in `main.py` depending on the selected exercise in the GUI.

```
def process_landmarks(self, landmarks, mp_pose):
    hip = landmarks[mp_pose.PoseLandmark.RIGHT_HIP.value]
    knee = landmarks[mp_pose.PoseLandmark.RIGHT_KNEE.value]
    ankle = landmarks[mp_pose.PoseLandmark.RIGHT_ANKLE.value]

    #ensures camera can confidently see user's leg
    if hip.visibility > 0.5 and knee.visibility > 0.5 and ankle.visibility > 0.5:
        hip_coord = [hip.x, hip.y]
```

```

        knee_coord = [knee.x, knee.y]
        ankle_coord = [ankle.x, ankle.y]

        raw_knee_angle = self.calculate_angle(hip_coord, knee_coord, ankle_coord)

        self.current_angle = self.euro_filter(time.time(), raw_knee_angle)

        self.update_fsm(self.current_angle)

    return True

#landmarks out of frame
return False

def update_fsm(self, knee_angle):
    if self.current_state == 'IDLE':
        if knee_angle < 160.0:
            self.current_state = 'FLEXING'
            self.min_angle_reached = knee_angle
            self.rep_quality = "In Progress"

    elif self.current_state in ['FLEXING', 'TARGET']:
        if knee_angle < self.min_angle_reached:
            self.min_angle_reached = knee_angle

    if self.current_state == 'FLEXING' and knee_angle <= self.target_angle:
        self.current_state = 'TARGET'

    if knee_angle > self.min_angle_reached + 15.0:
        self.current_state = 'STRAIGHTENING'

    #calculate how far off they were from their specific target
    angle_diff = self.min_angle_reached - self.target_angle

    if -4.0 <= angle_diff <= 7.0:
        self.rep_quality = "GOOD"
    elif (-7.0 <= angle_diff < -4.0) or (7.0 < angle_diff <= 10.0):
        self.rep_quality = "OKAY"
    elif angle_diff < -7.0:
        self.rep_quality = "BAD (Too Deep)"
    else:
        self.rep_quality = "BAD (Too Shallow)"

    elif self.current_state == 'STRAIGHTENING':
        if knee_angle > 160.0:

```

```

self.current_state = 'IDLE'
self.rep_count += 1
self.rep_history.append(self.min_angle_reached)

```

Figure 7: Example usages of the process_landmarks and update_fsm functions.

summary_screen.py: This file utilizes the Tkinter Python GUI library to show the patient's their exercise results after each set. Matplotlib is a Python library that is used to draw the angle achieved at each rep of the user's set, which is shown in Figure 7. After each set, a graph is shown to screen showing the target angle, and the angle that each repetition hit. This graph is then cleared, and is shown after the next set, repeating for all sets.

display_utils.py: All of the text that is shown on screen originates from this file. Two helper functions are used, draw_exercise_overlay and draw_warning. The first function is used while the user is in frame, doing their exercise. It shows the reps, state (from the Finite State Machine), angle of the exercise, and says whether the rep quality was good, bad, or okay. For draw_warning, this is what alerts the user that they cannot be seen in frame. This is shown when MediaPipe has less than 50% confidence it can see one of the user's main joints, and the code for that is in the process_landmarks method shown in Figure 7.

oneEuroFilter.py: The One Euro Filter [12] is a low-pass filter that is used to smooth noisy data streams. In this project, the skeletal tracking coordinates can include noisy data since the patient will always be moving, not allowing the program to get a great read of the joints every frame. The One Euro Filter is used to adjust its smoothing intensity based on how fast the user is moving, which eliminates jitter when the user is standing still and reduces lag when the user is moving.

The math behind the filter is as follows: a new, clean data point (x_{clean}) is calculated by blending noisy data (x) with the previous frames clean data (x_{prev}), which is governed by a smoothing factor (α):

$$\text{Formula - } x_{\text{clean}} = \alpha(x) + (1 - \alpha)(x_{\text{prev}})$$

$$\text{Related code - } \text{return } a * x + (1 - a) * x_{\text{prev}}$$

Where the One Euro Filter excels is how it calculates the cutoff frequency (f) which generates the α factor used in the previous step. The code included is from the oneEuroFilter.py file. Since the cutoff frequency scales linearly with the absolute speed ($|\text{speed}|$) of the user's movement, the cutoff frequency can be calculated as follows:

$$\text{Formula - } F = (\text{minimum } f) + \beta(|\text{speed}|)$$

$$\text{Related code - } \text{cutoff} = \text{self.min_cutoff} + \text{self.beta} * \text{abs}(\text{dx_hat})$$

Where β is the velocity scaling factor. When the speed approaches zero, the cutoff frequency drops to minimum f which triggers the intense filtering to get rid of the jitter. When the speed spikes, β rapidly increases the cutoff frequency which allows high-frequency data to go through the filter so there is no lag. To find the smoothing factor, the paper uses this formula:

$$\text{Formula} - 1 / (1 + (1 / (2\pi * f * T)))$$

Related code - $r = 2 * \text{math.pi} * \text{cutoff} * t_e$, followed by $\text{return } r / (r + 1)$

T denotes the specific time step the alpha value is found at. The top and bottom were both multiplied by r to get rid of the complex fraction. These are the key formulas used in the file. The combination of these formulas help reduce jitter or lag, depending on how quickly the user is moving.

Experiments

The software is started when a user enters “python main.py” while they are in the PercePT directory on their laptop. If the idea were to be explored by a larger company, this process would change since it is complex for a typical user, but this is perfect for what the first version of PercePT is trying to accomplish. As shown in Figure 6, a menu pops up with a few different options that are meant to happen in sequential order:

1. The user enters their target angle which will be prescribed by their physical therapist. Say a patient has a rotator cuff injury and needs to perform lateral raises to bring back range of motion to their arm. They obviously can't raise their arm straight above their head, and at the start, they can only raise it a certain amount. A physical therapist will take note of this and give the patient a certain degree angle that the user should hit so their arm does not get too far away from their body. As shown in Figure 20, the patient is raising their arm at a 48-degree angle, which shows that the system can keep track of where the patient needs their arm to be.
2. The target repetitions and target sets are entered by the user, signifying how many iterations of the exercise they want to accomplish. This number will also be given by their physical therapist and should never be decided by the patient. After each set, no matter how many sets they enter, there will be a post-set summary screen that shows the patient their results.
3. Since each patient will have a different amount of time they need to get in position for their set, they can change how long it takes until the camera starts tracking their movements. The timing is specified in seconds, as shown in Figure 6. The following image, Figure 9, shows the number on the screen counting down as the user sets up their screen:



Figure 9: Screen showing the countdown to the exercise.

4. A patient will get to select the exercise that the physical therapist wants them to do. As with the previous options, the exercise will always be decided by the physical therapist.

As it was mentioned for the previous three options, the physical therapist will be providing all the information for setting up the exercises. This is to further enhance the idea that the physical therapist still organizes the workouts but can watch the user conduct their exercises from their own home.

Once the user selects their exercise, it takes them straight to the live camera feed, where they can see themselves with the MediaPipe skeletal tracking following their joints. Since the user may need time to set up in front of the camera, a dynamic countdown is provided that allows the user to choose how long they have left to get set, which was shown previously in Figure 6. On the live-feedback screen, the user can see their reps, the state they are in, the degree angle of their exercise, and feedback on whether the repetition was bad, good, or okay. The following images show how a repetition is conducted in the system:



Figure 10: Patient standing in the idle state.

1. The user starts in the idle state. As it is shown in the top left of the image, the reps, state, angle, sets, and previous repetition's success are shown. The skeleton is drawn on the patient's body, marking the 33 respective joints. It is important to note that MediaPipe is intelligent enough to decipher the patient's body from the background, even though the background is not a solid color.

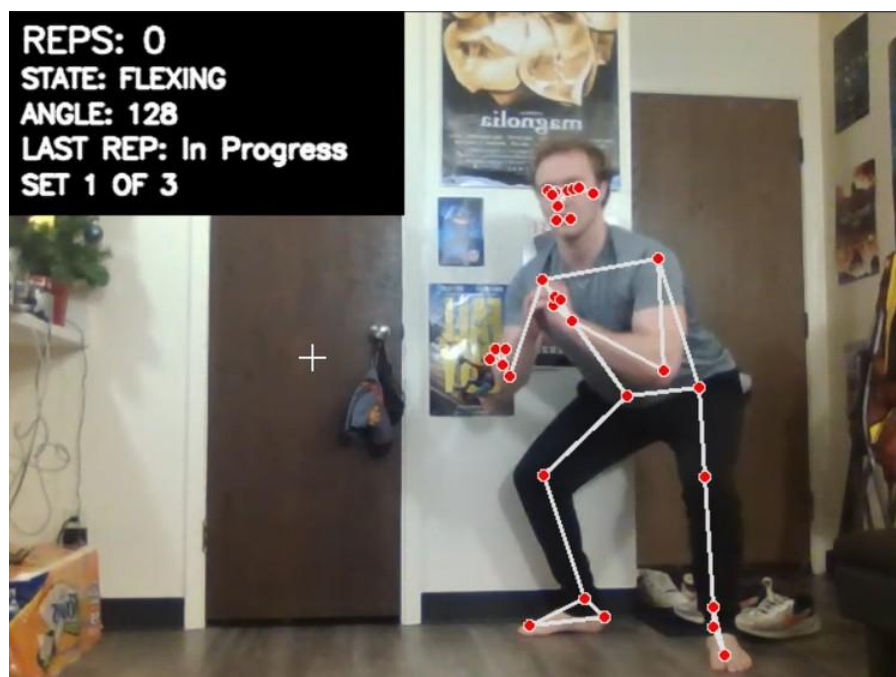


Figure 11: Patient descending into the flexing state.

2. As the patient descends towards the target state, they enter the flexing state, signaling that the patient is changing the angle of their limb/body in their exercise. Even though they are turned at a 45-degree angle, the MediaPipe skeletal tracking has no issues tracking their body.



Figure 12: Patient in the target state.

3. When the exercise gets close to the target angle, the Finite State Maching will enter the target state. In the examples for Figures, 10, 11, and 12, the target angle was set at 100 degrees. When the user reaches the target state, the deepest degree angle they hit before they enter the straightening state is set as the angle they achieved for that repetition. For example, in Figure 12, if the patient starts standing up, their final target angle for that repetition will be set as 107, and they will be given a good/bad/okay message accordingly.



Figure 13: Patient in the straightening state.

4. Once the user enters the straightening state. They are given the quality of their last rep with the associated color of green, yellow, and red for good, okay, and bad respectively. The reps counter does not increase in this phase, it increases once the user has officially reached the idle state from the straightening state.

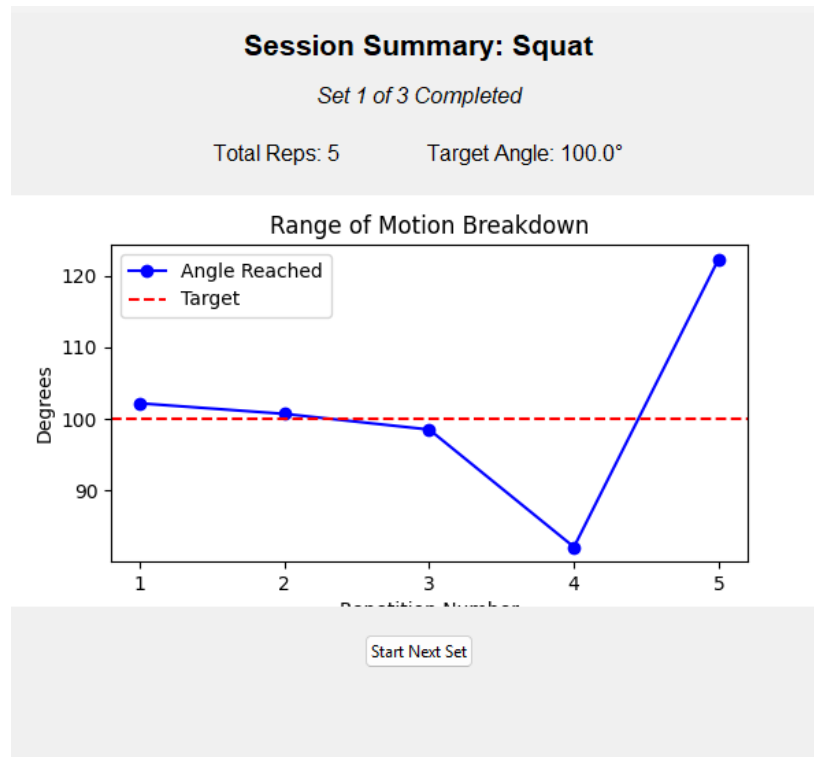


Figure 14: Post-Session Summary Screen

- Once the patient has completed all of their repetitions for a set, they receive a post-session summary recap of how close they got the target angle for each repetition. As it is shown in the legend in Figure 14, the blue line represents the angle reached for each repetition, and the red dotted line shows that target angle that was entered by the patient. The graphic also shows the sets completed, exact target angle value, and total reps. This screen is helpful for the patient, so they have a general idea of how low they squatted, and how close it was to their target.

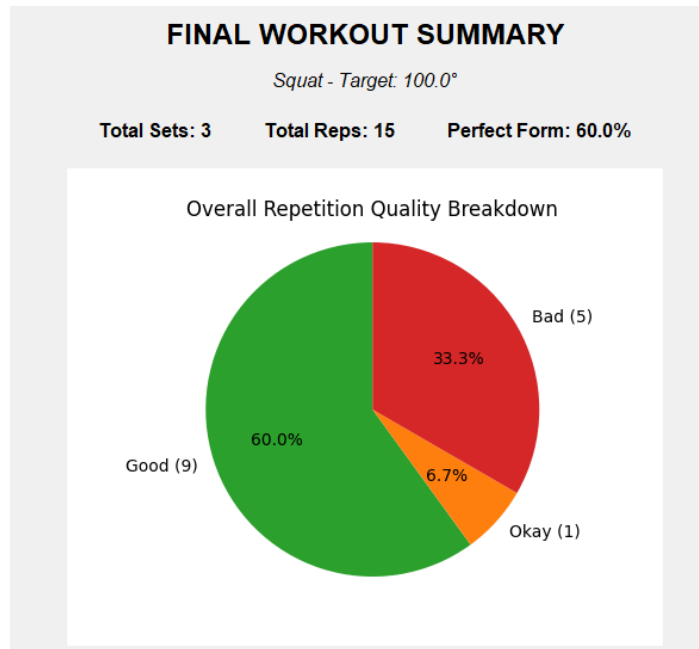


Figure 15: Final Workout Summary Screen

6. Once the workout is over, meaning that all sets and repetitions have been completed, a pie chart is shown to screen showing how well all of the patient's reps went. In Figure 15, it shows that the user had 9 good reps, 5 bad reps, and 1 okay rep, posting a perfect form score of 60%. It also shows the total sets and reps completed, with the exercise and target angle as well. In a commercial environment, this would be used by the physical therapist to see how the patient is doing and recommend a more or less severe angle depending on their progress.

At any point, if the user wants to quit their workout, they can press the 'q' button on their keyboard which will take them straight to the summary for that set, and then the Final Workout Summary Screen letting them know what they have accomplished in that session. The following figure is an example of what that screen looks like:

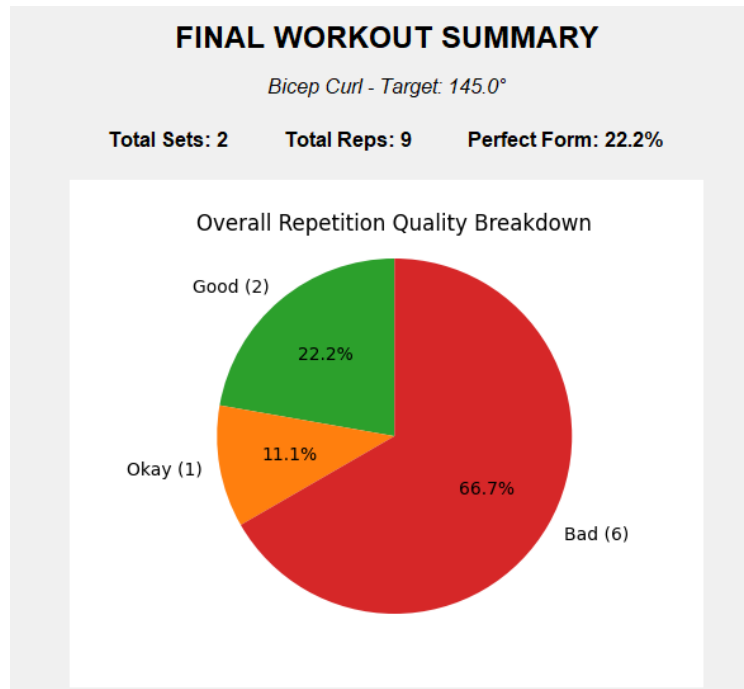


Figure 16: Final Workout Summary Screen when user quits application early.

At the start of this session, it was specified to have three total sets and six reps for every set. It is shown in Figure 16 that the patient had made it to the second set, and conducted three reps within that set before they decided to quit the application. This will be a rare scenario since the patient knows how important it is that they finish the sets and reps that the physical therapist gives them, but it allows them to quit under extreme circumstances.

The other four exercises used the exact same methodology as squat, which includes front lunge, lateral raise, standing knee bend, and bicep curl. The following images, Figures 18, 19, 20, and 21, are meant to show what each of those exercises look like in the flexing, target, or straightening state.

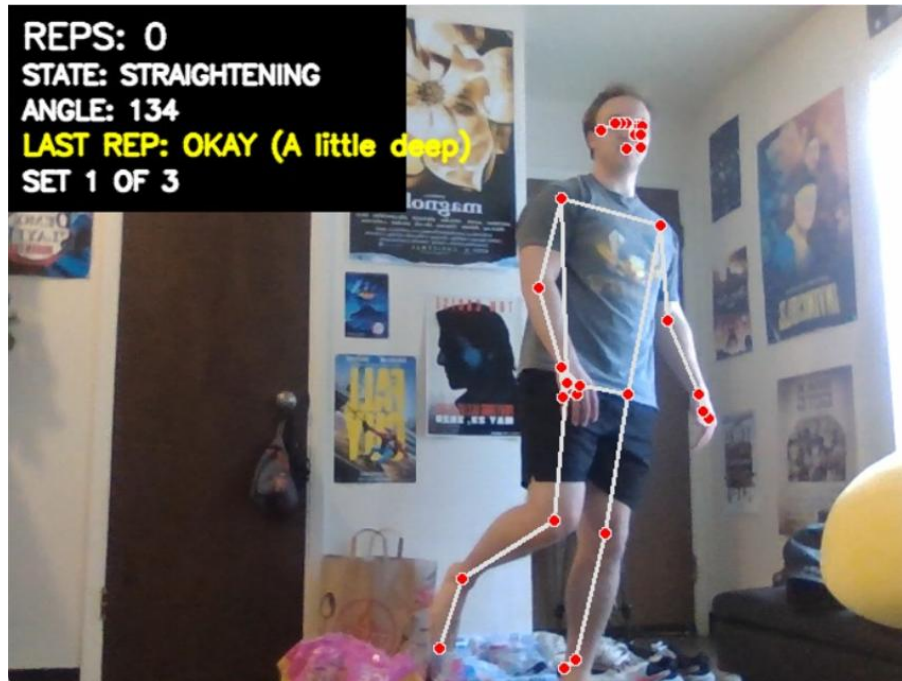


Figure 18: Straightening state of the knee bend exercise.

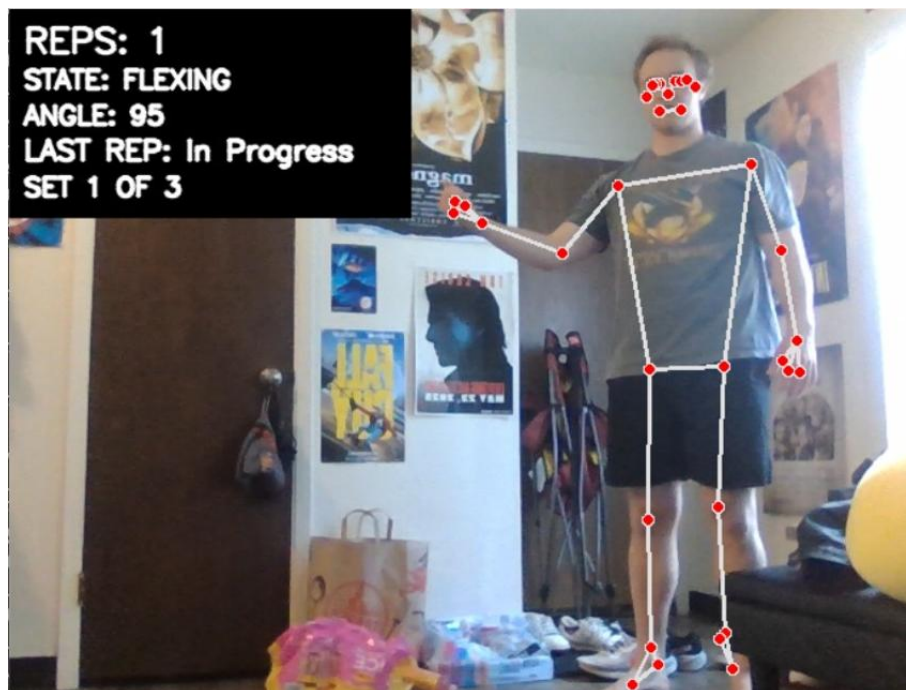


Figure 19: Flexing state of the bicep curl exercise.

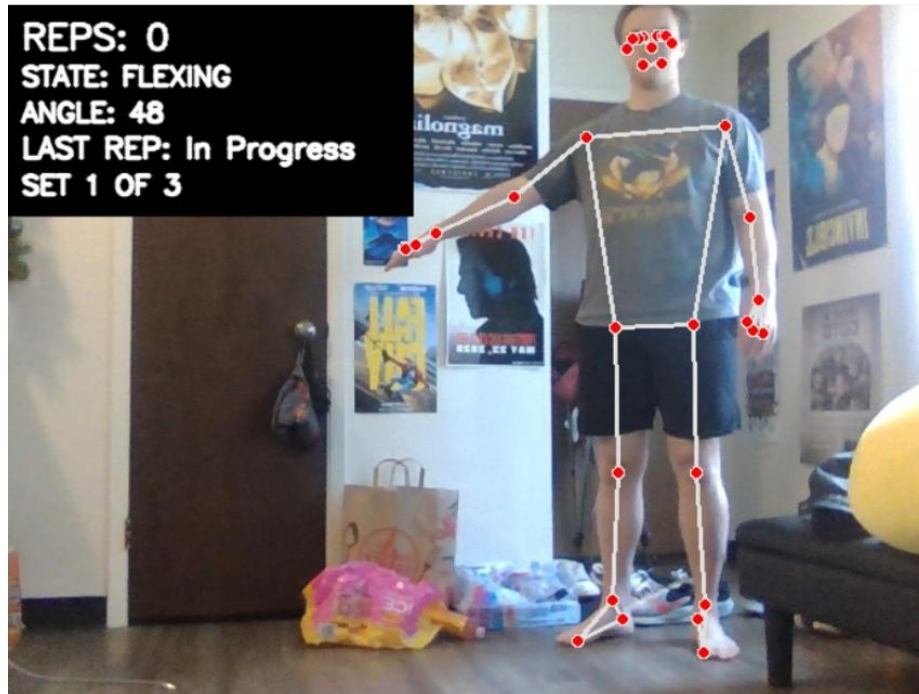


Figure 20: Flexing state of the lateral raise exercise.

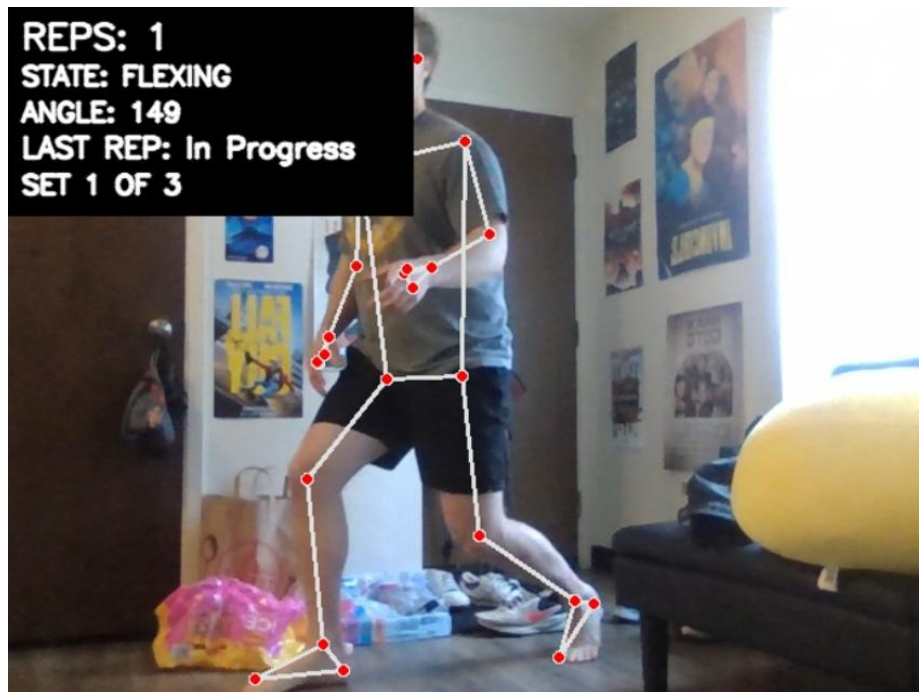


Figure 21: Flexing state of the front lunge exercise.

Results

There were no flaws in the system design from what was proposed in the beginning to the final product in the end. The primary focus for this project was the live video. The system is meant to

read a human body, use MediaPipe to track their most important joints, grade their exercise performance in real time, and report this information back to them both during their set, after their set, and at the end of their workout. The One Euro Filter was successful in eliminating the jitter that came from the MediaPipe landmarks. Importantly, the system was able to run the live OpenCV feed, and use the BlazePose neural network with MediaPipe to get the coordinate data. The Finite State Machine was then utilized to structure the workouts. All of these working parts were ran on a laptop, proving that PercePT is a lightweight application that can run given ubiquitous hardware. Throughout testing and implementation, there was no noticeable lag in the system.

When in frame, the system had great success with viewing the patient. When starting this project, a big topic I believed that I would have to explore was how well MediaPipe and BlazePose can work together to read the 33 major joints it selected on the body. When testing throughout the project, I was in multiple different environments, all with different backgrounds, and did not struggle with being recognized by MediaPipe. There was only one scenario that was difficult for MediaPipe:

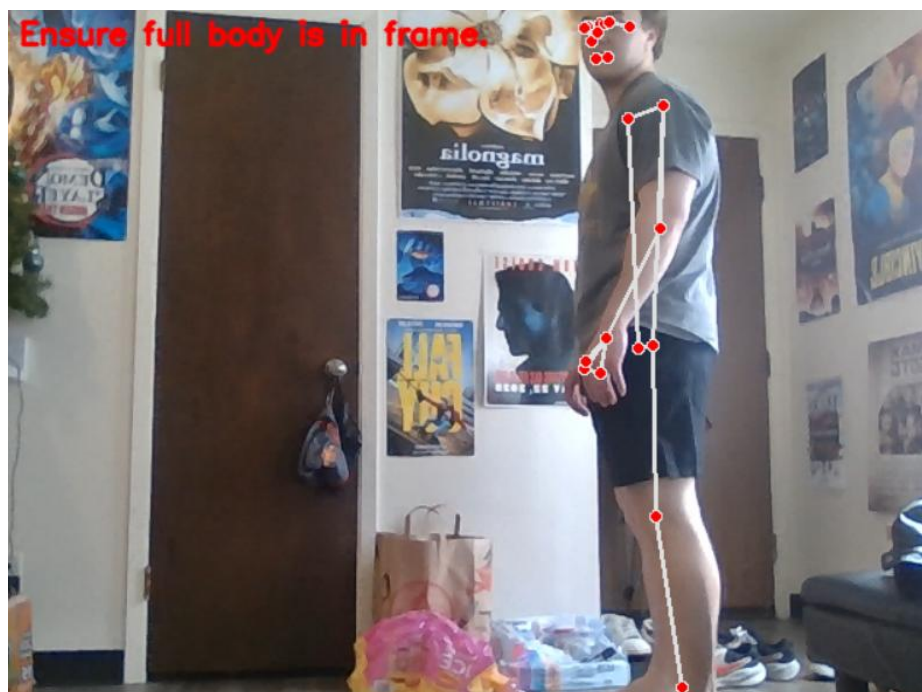


Figure 17: Body not-in-frame error.

Whenever the patient is turned sideways, MediaPipe struggles to locate all of the joints. This makes sense, since for example, the right leg is completely hidden behind the left leg, so MediaPipe cannot confidently say that it knows where the right leg is at. For an exercise such as the squat, it was unknown whether this would be an issue or not since it is assumed that seeing the squat from a side angle is the easiest way to judge the angle of the legs. This is an easily avoidable issue; all the patient has to do is look at the camera head on or at a 45-degree angle so MediaPipe can recognize their hidden leg. The Finite State Machine did an incredible job of staying consistent in its calculations. This made sense since the program just had to read the calculated angle and use if-else statements to determine the rep's quality.

Room for Improvement

Originally, the goal of this project was to also implement a web interface that would allow a patient and physical therapist to communicate without having to meet in person. The physical therapist could give feedback on the patient's exercises at home so the patient knows if they are executing the exercise to their greatest potential. It is important to note that the goal was never to replace the sessions that the physical therapist conducts with the patient. Since human physiology improves every day through strength building exercises, the therapist will not require the patient to show up to the office every day to watch their exercises. PercePT provides the ability to give feedback to the patient from a remote location. The implementation of the website would be the final step in the process for allowing this communication between the two parties. This would require a significant amount of parts. For example, a database to hold user information, login/password system, and calendar system to help the patient remember their exercise. While the website itself is a big addition, it is imperative that it is implemented well. Anytime an application deals with patient data, hackers are likely to target these applications for data breaches. With an application that uses a variety of libraries, the versioning software will constantly need to be worked on to ensure there are no security risks. Python 3.10.20 is being used to run this project which works well with the given APIs, but with Python being on version 3.13 already, this version is roughly three years behind [20].

Discussion

The primary goal of PercePT was to add a whole other element to the physical therapist to patient relationship by allowing a patient to do their prescribed exercises at home, getting real time feedback into whether they were doing their exercises effectively. This project accomplished everything it set out to do in relation to this course. As it was mentioned, it was not possible to have the website done given the time frame, but a Python generated GUI using the Tkinter library was used, resulting in an easy-to-understand user interface. The end-to-end testing provided by the five exercises given, primarily squat which was the exercise documented in this report, proved that this application is scalable to allow new exercises. This idea is further proved by the Object-Oriented Programming principles used in the code to allow new classes to be easily implemented. The combination of the MediaPipe framework, One Euro signal filter, and Finite State Machine showed that the system is highly capable of processing raw video data into feedback. Even though the experiments were conducted in the same environment with the same background, the diversity within the background proves this system can be run in any environment.

The biggest shortcoming was MediaPipe's lack of ability to recognize joints when the patient turns sideways. This makes sense, since joints overlapping with each other on a 2-dimensional plane makes it almost impossible to identify which is which. It is important to keep in mind that MediaPipe is an open-source solution to this problem provided by Google. It does a phenomenal job of tracking joints and is the open-source standard, so if there are commercial products that a company would want to use for this system, they can implement the APIs accordingly. Even then, to fix this issue, it requires the user to understand that they can't be faced sideways, and the timer given to the user to allow them to get set before the system starts counting needs to be utilized. Besides this, the project did not seem to have any issues. Counting and exercise tracking worked

extremely well thanks to the Finite State Machine, MediaPipe skeletal tracking, and angle calculation.

Conclusion

PercePT is a low cost, lightweight application that excels at live-video feedback for physical therapy patients. While Kemtai has low hardware restrictions for a high-quality software product, it is still a closed-source software that relies on many external API's to function. PercePT's biggest benefit is that it is heavily built on the idea of giving quick, real-time feedback, all while being a lightweight application. If the web application were to be continued, knowing that a physical therapist sent them the video directly lets the patient feel as if the therapist is watching them, even though they aren't together in person. Also, PercePT would ideally be sold at the enterprise level, not costing the patient's any money. Before Kemtai adapted a B2B Enterprise model, they were originally pricing their subscription plan at \$19/month [16]. PercePT will never charge anybody a subscription, since the patient to therapist connection is what makes it special. Every patient that uses PercePT will have it recommended to them by a therapist. PercePT's rigorous implementation for the live-video feed allows it to be easily implementable in a web interface, making it the first step in the process for an application that can change the physical therapy market for the better.

In the future, the addition of a user interface will make this project come full circle. As it stands, PercePT is a codebase that must be ran using Python. Not only would the user interface for a web application need work, but the UI for the live video feed could also be improved. It currently includes the bare minimum text needed for the patient to be aware of their exercise, but a cleaner look would benefit it. Within the website, a patient dashboard could be implemented containing information such as completed workouts, future workouts, and see a ratio of how successful their reps have been up to date. The user interface as a whole has potential, but these are a few key areas that were going to be implemented if there was more time for the project. Also, for those with impaired vision, an auditory cue system could have been implemented if there was more time allocated for the project as well.

REFERENCES

- [1] Salamon, M. *Physical therapy: Myths vs. reality*. <https://www.health.harvard.edu/staying-healthy/physical-therapy-myths-vs-reality> (March 4th, 2026).
- [2] American Physical Therapy Association. *A Physical Therapy Profile: Demographics of the Profession, 2021-2022*. American Physical Therapy Association. 2023.
- [3] Harris Williams. *Physical therapy market overview Q2 2022*. Harris Williams. 2022.
- [4] Kemtai. *About*. <https://kemtai.com/about/> (March 6, 2026).
- [5] Kemtai. *Computer Vision Rehab*. <https://kemtai.com/product/computer-vision-rehab/> (March 6, 2026).
- [6] Kemtai. *Range of Motion*. <https://kemtai.com/product/range-of-motion/> (March 6, 2026).
- [7] Kemtai. *Exercise Library*. <https://kemtai.com/product/exercise-library/> (March 6, 2026).
- [8] Heo, S., Choi, T., and Choi, W. Clinical Validation of an On-Device AI-Driven Real-Time Human Pose Estimation and Exercise Prescription Program; Prospective Single-Arm Quasi-Experimental Study. *Healthcare*. 14. 4. p. 482. 2026.
- [9] Dill, S. *et al.* Accuracy Evaluation of 3D Pose Reconstruction Algorithms Through Stereo Camera Information Fusion for Physical Exercises with MediaPipe Pose. *Sensors*. 24. 23. p. 7772. 2024.
- [10] Wan Noor Asmawi, W., and Kumaresan, S. Real-Time Posture Monitoring for Effective Exercise Using MediaPipe Python. *Journal of Informatics and Web Engineering*. 5. 1. pp. 236–251. 2026.
- [11] Yang, H., Wang, Y., and Shi, Y. Rehabilitation Training Evaluation and Correction System Based on BlazePose. In *Proceedings of the 2022 IEEE 4th Eurasia Conference on IOT, Communication and Engineering (ECICE)*. (2022). DOI: 10.1109/ECICE55674.2022.10042886.
- [12] Casiez, G., Roussel, N., and Vogel, D. 1€ Filter: A Simple Speed-based Low-pass Filter for Noisy Input in Interactive Systems. In *Proceedings of the 30th Conference on Human Factors in Computing Systems*. pp. 2527–2530. (Austin, TX, 2012). DOI: 10.1145/2207676.2208639.
- [13] Google. *Pose landmarker*. https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker (March 6, 2026).
- [14] Mishra, Y. *et al.* AI Human Fitness Tracker using Computer Vision with MediaPipe. *International Journal for Research in Applied Science and Engineering Technology*. 13. 5. pp. 1659–1669. 2025.

- [15] Biebl, J. *et al.* App-Based Feedback for Rehabilitation Exercise Correction in Patients With Knee or Hip Osteoarthritis: Prospective Cohort Study. *Journal of Medical Internet Research*. 23. 7. p. e26658. 2021.
- [16] Fink, T. *Kemtai Adaptive Home Exercise Platform review*.
<https://www.laptopmag.com/reviews/kemtai-adaptive-home-exercise-platform> (March 6, 2026).
- [17] NumPy. What is NumPy? <https://numpy.org/doc/2.3/user/whatisnumpy.html> (March 6, 2026).
- [18] OpenCV. *OpenCV: Introduction*. <https://docs.opencv.org/4.x/d1/dfb/intro.html> (March 8, 2026).
- [19] Balogh, R., and Obdrzalek, D. Using Finite State Machines in Introductory Robotics: Methods and Applications for Teaching and Learning. In *Proceedings of the 9th International Conference on Robotics in Education*. pp. 85–91. (2018). DOI: 10.1007/978-3-319-97085-1_9
- [20] Python Software Foundation. Python documentation by version | Python.org.
<https://www.python.org/doc/versions/> (April 19, 2026).
- [21] Python Software Foundation. tkinter - Python interface to Tcl/Tk.
<https://docs.python.org/3/library/tkinter.html> (April 19, 2026).
- [22] Python Software Foundation. time - Time access and conversions.
<https://docs.python.org/3/library/time.html> (April 19, 2026).
- [23] Matplotlib Development Team. Matplotlib - Visualization with Python.
<https://matplotlib.org/> (April 19, 2026).